

8.12模拟面试

1.应用层的协议：HTTP, HTTPS, FTP, DNS, DHCP, Telnet, Smtpt, SSH

FTP：文件传输协议。DNS：域名解析协议，域名解析成IP地址。Smtpt：邮件传输协议

2.数据链路层的作用：相邻节点间通过物理链路可靠传输数据；

1. 对网络层数据进行帧的封装与解封装，界定传输单位并标识收发节点；
2. 通过 CRC 等校验机制进行差错控制，检测并处理传输错误；
3. 采用滑动窗口等协议实现流量控制，避免接收方因处理不及而丢失数据；
4. 解决多节点共享介质的冲突问题，如以太网的 CSMA/CD 等介质访问控制机制。

3.三次握手的原因：

TCP 建立连接时，通过三次握手能防止历史连接的建立，能减少双方不必要的资源开销，能帮助双方同步初始化序列号。序列号能够保证数据包不重复、不丢弃和按序传输。

不使用两次握手和四次握手的原因：

两次握手：无法防止历史连接的建立，会造成双方资源的浪费，也无法可靠的同步双方序列号；

四次握手：三次握手就已经理论上最少可靠连接建立，所以不需要使用更多的通信次数。

4.粘包和拆包

粘包：指发送方发送的多个独立数据包，在接收方被合并成一个数据包接收的现象。

原因：TCP 是面向字节流的协议，不维护消息边界，仅按字节序列传输，接收方无法直接区分多个消息的分隔。

发送方发送数据过快，或数据量较小（小于 TCP 缓冲区大小）时，TCP 会将多个小数据包合并为一个大的 IP 数据包发送（Nagle 算法优化），减少网络开销。

接收方缓冲区未满，会暂存多个数据包，待缓冲区数据积累到一定量再一次性交给应用层。

拆包：指发送方发送的一个大数据包，在接收方被拆分成多个小数据包接收的现象。

原因：当发送的数据量超过 TCP 最大报文段长度（MSS）或 IP 层最大传输单元（MTU）时，TCP 会将数据拆分后分批次传输。

网络拥塞时，TCP 可能将未发送的大数据包拆分，优先传输部分数据以适应网络状况。

5.MSL和TTL

MSL是 Maximum Segment Lifetime，报文最大生存时间，它是任何报文在网络上存在的最长时间，超过这个时间报文将被丢弃。因为 TCP 报文基于 IP 协议的

IP 头中有一个TTL字段，是 IP 数据报可以经过的最大路由数，每经过一个处理他的路由器此值就减1，当此值为0则数据报将被丢弃，同时发送 ICMP 报文通知源主机。

6.MSL与 TTL的区别: MSL的单位是时间，而 TIL是经过路由跳数。所以 MSL应该要大于等于 TIL消耗为 0 的时间，以确保报文已被自然消亡。

7.为什么需要 TIME_WAIT 状态？

主动发起关闭连接的一方，才会有TIME_WAIT 状态。需要 TIME_WAIT 状态，主要是两个原因：防止历史连接中的数据，被后面相同四元组的连接错误的接收:保证被动关闭连接的一方，能被正确的关闭;

8.TCP和UDP的区别？应用场景？

1. 连接：TCP是面向连接的传输层协议，传输数据前要先要简历连接。UDP是不需要连接，即刻传输数据
2. 服务对象：TCP是一对一的两点服务，即一条连接只有两个端点。UDP支持一对一、一对多、多对多的交互通信。
3. 可靠性：TCP是可靠交付数据的，数据可以无差错、不丢失、不重复、按序到达。UDP是尽最大努力交付，不保证可靠交付数据
4. 拥塞控制，流量控制：TCP有拥塞控制和流量控制机制，保证数据传输的安全性。UDP没有，即使网络非常拥堵了，也不会影响UDP的发送速率。
5. 首部开销：TCP首部长度较长，会有一定的开销，首部在没有使用选项字段时是20字节，使用了则会变长。UDP首部只有8个字节，并且是固定不变的，开销较小
6. 传输方式：TCP是流式传输，没有边界，但保证顺序和可靠。UDP是一个包一个包的发送，是有边界的，但可能会丢包和乱序。
7. 分片不同：TCP的数据大小如果大于MSS大小，则会在传输层进行分片，目标主机收到后，也同样在传输层组装TCP数据包，如果中途丢失了一个分片，只需要传输丢失的这个分片。UDP的数据大小如果大于MTU大小，则会在IP层进行分片，目标主机收到后，在IP层组装完数据接着再传给传输层。

9.拥塞控制

拥塞控制用于防止网络因数据包过多而陷入拥塞。目的是在保证网络稳定性的前提下，最大化数据传输效率。

TCP通过动态调整发送方的拥塞窗口来控制数据发送速率。拥塞窗口是发送方在未收到确认(ACK)的情况下，最多能发送的数据包数量，直接决定了数据注入网络的速度。

拥塞控制是根据网络拥塞状态动态调整cwnd——网络空闲时增大窗口以提高效率，网络拥塞时减小窗口以缓解压力。

TCP拥塞控制的四个阶段，不同阶段通过不同算法调整cwnd：

1. 慢启动：初始时cwnd较小（通常为1~2个最大报文段长度MSS），每收到一个ACK，cwnd就加倍（指数增长），快速适应网络初始容量。为避免窗口增长过快导致拥塞，设置慢启动阈值（ssthresh）。当cwnd达到ssthresh时，进入“拥塞避免”阶段。
2. 拥塞避免：cwnd超过ssthresh后，或网络处于稳定状态时。不再指数增长，而是每收到一个ACK，cwnd缓慢增加1个MSS（线性增长），逐步试探网络最大承载能力，避免突然拥塞。
3. 快重传：当网络出现拥塞信号时，TCP会调整cwnd和ssthresh，若通过重复ACK检测到丢包，执行“快速重传”+“快速恢复”；ssthresh设为当前cwnd的一半，cwnd设为ssthresh，然后进入拥塞避免阶段。若通过超时检测到丢包；ssthresh设为当前cwnd的一半，cwnd重置为初始值，重新进入慢启动阶段。
4. 快恢复：不直接进入慢启动，而是保持较高的cwnd继续发送数据，同时快速重传丢失的数据包，减少因拥塞导致的吞吐量下降。

10.RTT——往返时延

从发送端发送数据开始，到发送端接收到来自接收端的确认（接收端收到数据后便立即发送确认），总共经历的时延。也就是从发送数据到接收到ack的时延。

11.RTO——超时重传时间

TCP每发送一个报文段，就会对此报文段设置一个超时重传计时器。此时计时器设置的超时重传时间RTO应当略大于TCP报文段的平均往返时延RTT，一般可取 $RTO=2RTT$ 。

12.四次挥手全流程

1)主动方打算关闭连接，此时会发送一个TCP首部FIN标志位被置为1的报文，也即FIN报文，之后主动方进入FIN_WAIT_1状态

2)被动方收到该报文后，就向主动方发送ACK应答报文，接着被动方进入CLOSED_WAIT状态。

3)主动方收到被动方的 ACK 应答报文后, 之后进入 FIN_WAIT_2状态

4)等待被动方处理完数据后, 也向主动方发送FIN 报文, 之后被动方进入 LAST_ACK 状态。

5)主动方收到被动方的 FIN 报文后, 回一个ACK 应答报文, 之后进入 TIME_WAIT 状态

6)被动方收到了 ACK 应答报文后, 就进入了CLOSED 状态, 至此被动方已经完成连接的关闭

7)主动方在经过2MSL ($2 * \text{Maximum Segment Lifetime}$,两倍的报文段最大存活时间)一段时间后, 自动进入CLOSED状态, 至此主动方也完成连接的关闭

13.为什么是4次挥手不是三次?

被动方在收到FIN后, 要立刻回一个ACK, 将之前收到的数据都处理完再回复FIN包, 立刻回ACK是为了防止超时重传, 因为如果处理数据的时间过长就会触发超时重传机制。如果不处理数据的话是可以三次的。